Making fewer points mean more.

What building cloudForge taught me about C++, CUDA and honest performance claims.



A real recorded scan, displayed twice: original samples on the left; voxel centroids on the right. The scene remains recognizable while the point count falls.

A point-cloud frame can contain tens of thousands of XYZ samples. Repeating that work over a recording turns a small algorithm into a systems problem: how do I read the data, reduce it, keep the interface responsive and prove that acceleration still produces the right result?

I built cloudForge to explore that question with C++20 and an optional CUDA backend. The application loads recorded PCD frames, groups samples into voxels and replaces each occupied cell with its centroid. It then plays the original and simplified sequence side by side.

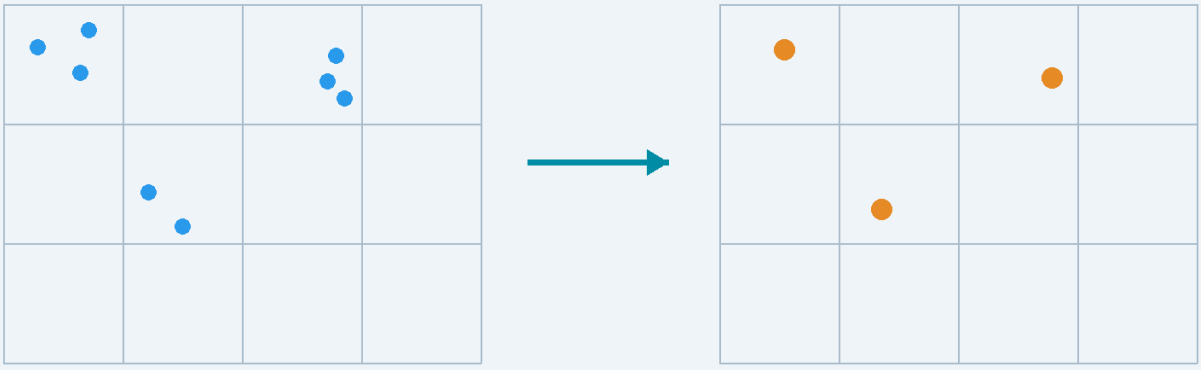
One real frame shrank from **55,518 points to 9,943** at voxel size 0.3 - an **82.09% reduction**. That is the core idea: fewer samples, while preserving the broad shape. It is not a guarantee that every fine geometric feature survives.

Start with a rule you can explain.

One voxel contains many samples, but contributes only one output point.

One occupied voxel. One output centroid.

A 2D slice illustrates the same rule used along X, Y and Z.



INPUT: samples grouped by voxel

OUTPUT: one representative per group

For voxel size s, the key is floor(x/s), floor(y/s), floor(z/s). Points with the same key belong to the same group. Their arithmetic mean becomes the output point. Two samples at (0.1, 0.2, 0.3) and (0.2, 0.3, 0.4) produce the centroid (0.15, 0.25, 0.35) when s = 1.

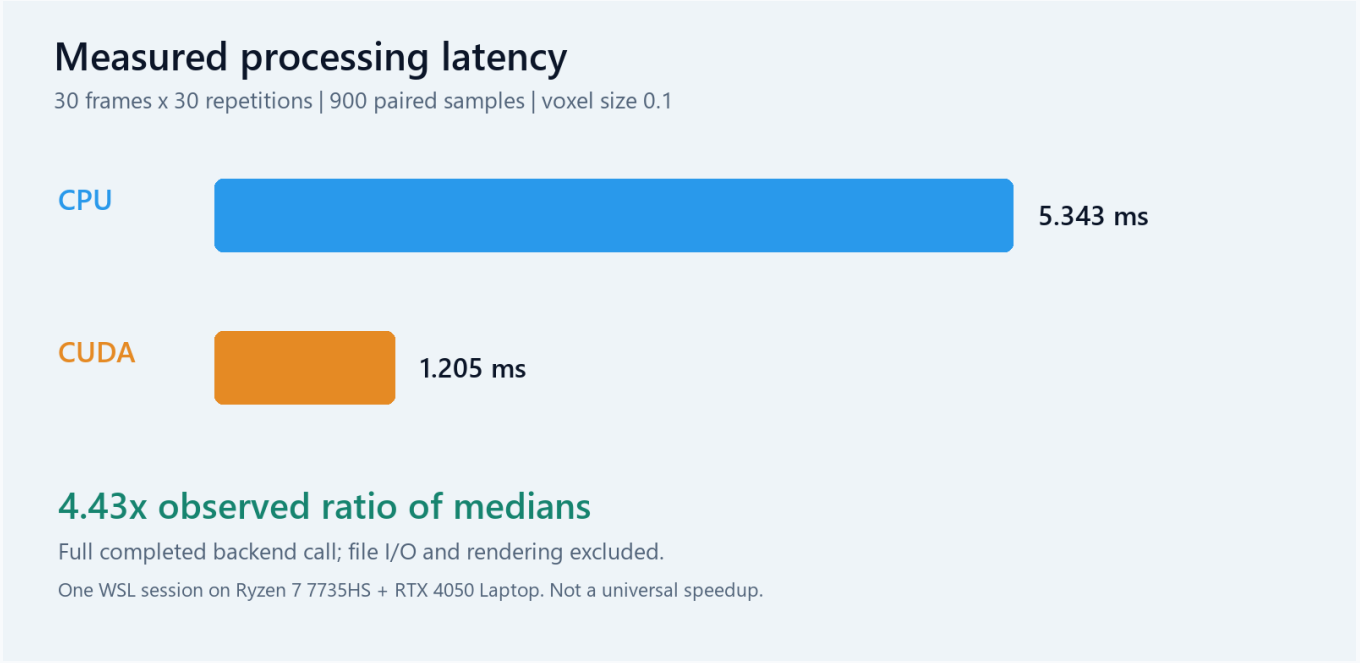
The first important detail was negative coordinates. Truncating -0.1 to zero would put it in the wrong cell. Floor puts it in cell -1. Exact boundaries, empty input, duplicates, NaN, infinity and integer range limits became tests rather than assumptions.

I made the CPU reference deterministic for a fixed input order and kept its output sorted by voxel key. Only after testing that baseline did I introduce CUDA. This gave me something trustworthy to compare against when parallel reduction changed floating-point rounding.

Lesson: correctness is not a screenshot of a similar-looking cloud. It is a defined contract, explicit edge cases and a repeatable comparison of counts and coordinates.

The GPU did not win automatically.

Launches, transfers, allocation and synchronization can cost more than the useful work.



An early recorded run at voxel size 0.1 measured CPU median 5.905 ms and CUDA median 7.326 ms. Moving work to a GPU was not enough. I replaced a repeated-allocation, comparison-sort path with reusable buffers, collision-free packed keys, CUB radix sorting and reduce-by-key. A wide-range fallback preserves signed keys. The CPU baseline remains the single-threaded ordered-map reference, not an optimized parallel CPU implementation.

In the new session, 30 real frames with 30 repetitions each produced CPU median 5.343 ms and CUDA median 1.205 ms: a 4.43x observed ratio. All 900 comparisons passed. The earlier and current sessions were not controlled side-by-side, so this is not an ablation study or a universal acceleration claim.

The benchmark pauses rendering, fixes the warm-up order and alternates measured order. It includes host validation, required allocation, transfers and completed GPU output. It excludes file loading, warm-up and drawing. CSV rows preserve every sample; graphs expose medians and variation instead of presenting one changing ratio as certainty.

Lesson: a performance number needs its workload, hardware, timing boundary and correctness evidence. My test laptop used a Ryzen 7 7735HS and RTX 4050 Laptop GPU. Clocks, thermals and background activity still vary.

A project becomes useful at its edges.

The application around the kernel made the work worth doing.



The file browser initially stalled on repeated Windows-drive metadata checks. Caching entry metadata and moving scans to a worker made navigation responsive. A long-lived worker also kept the CUDA workspace alive. Original and simplified frames now appear as a matched pair, with synchronized pan, orbit and zoom.

Recorded playback uses one processing call per frame and offers pause, stepping, restart, looping and speed controls. Benchmarking has its own resizable window, but reuses the loaded input. Running it again recalculates and saves a fresh report automatically. These are small interface decisions with large effects on whether the result is understandable.

I also tested a native Windows CPU build: all 35 tests passed and 60 real frames played successfully. The WSL CUDA build passed 41 tests. Native Windows CUDA remains unverified because the required Windows toolkit/toolset is missing. I would rather state that boundary than imply platform coverage I have not demonstrated.

What I would build next: a point-count scaling study, bounded prefetch for smoother playback, per-stage GPU profiling and native Windows CUDA qualification. The next optimization should come from measurement, not from choosing the most impressive-looking ratio.

Screenshots: actual cloudForge app captures. Data: Oxford Spires Dataset, Tao et al., official project, CC BY-NC-SA 4.0. This illustrated article shares that media license; cloudForge source code remains MIT. Benchmark evidence and hardware details accompany the technical report.